Assignment 2: Paper 2

On Large-Batch Training for Deep Learning:
Generalization Gap and Sharp Minima

Jireh Katebe

CS 6362 Advanced Machine Learning, Fall 2026
22 October 2026

1 Problem statement and understanding of the methodology

1.1 The problem and intuition

The authors investigate why training with larger batches can lead to poorer generalisation [1]. Larger batches allow more computation to happen in parallel, which could help speed up training. But small-batch and large-batch methods can reach similarly low training losses while performing differently on unseen data. The paper asks whether the shape of the loss around the solutions reached during training helps explain this difference.

Near a sharp solution, a small change in some directions through the model's weights can increase the loss quickly. Near a flat solution, the loss changes more gradually over the neighbourhood being examined. Small batches produce noisier gradient estimates, which the authors suggest help the optimiser escape sharp regions. Larger batches produce less noisy estimates and may settle in sharper regions. The experiments investigate whether these sharper solutions also have poorer test performance.

1.2 Mathematical formulation

Let $S = \{(x_i, y_i)\}_{i=1}^n$ contain n training examples drawn from a distribution $\mathcal{D}$, with labels $y_i \in \{1, \dots, K\}$. The network has parameters $w \in \mathbb{R}^P$ and predicts class probabilities $p_w(y \mid x)$. With cross-entropy $\ell_i(w) = -\log p_w(y_i \mid x_i)$, the empirical and expected losses are

$$L_S(w) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(w), \qquad R(w) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[-\log p_w(y \mid x)].$$

Training minimises L_S, while a held-out test set estimates performance on unseen examples. Following the minibatch formulation in the lectures [2], a batch B_t gives the gradient estimate

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_w \ell_i(w_t).$$

The paper uses Adam to update w_t from these gradients, with the learning rate controlling the size of the updates. The averaging argument in Lecture 6 explains why independent gradient noise decreases as batch size increases. Batch normalisation (BN) and dropout complicate this idealised calculation because the gradients also depend on batch statistics and randomly dropped units.

Let w_{SB} and w_{LB} denote the small-batch and large-batch solutions. The problem is to understand why similar empirical losses can accompany different test performance, and whether local sensitivity of the loss helps account for that difference.

1.3 How the authors test it

The experiments use Adam for both methods, with 256 examples per small batch and 10% of the training set per large batch. Each setup is repeated from five random initialisations. The assigned Figure 3 panels are F1, a fully connected network on MNIST, and C1 and C2, shallow and deeper convolutional networks on CIFAR-10.

Figure 3 selects a pair of trained solutions and evaluates the path

$$w(\alpha) = (1-\alpha)w_{\mathrm{SB}} + \alpha w_{\mathrm{LB}}, \qquad \alpha \in [-1, 2].$$

At $\alpha = 0$ the weights equal the small-batch solution; at $\alpha = 1$ they equal the large-batch solution. Values outside $[0, 1]$ extrapolate beyond the endpoints. The models are already trained, so α specifies where to evaluate the loss along this path.

The horizontal axis shows α. The left vertical axis shows mean cross-entropy and the right shows classification accuracy. Solid curves use training data and dashed curves use test data. In the paper, the loss rises more steeply around the large-batch endpoint, while the small-batch endpoint lies in a broader low-loss region.

To examine directions beyond this path, the authors define the sharpness measure in their equation (4):

$$\phi_{w,L_S}(\epsilon, A) = 100\,\frac{\max_{z \in C_\epsilon} L_S(w + Az) - L_S(w)}{1 + L_S(w)},$$

$$C_\epsilon = \{z \in \mathbb{R}^r : |z_j| \le \epsilon(|(A^+ w)_j| + 1),\ j = 1, \ldots, r\}.$$

Here $A \in \mathbb{R}^{P \times r}$ selects the search directions and A^+ is its pseudoinverse. The authors approximate the maximum using $A = I_P$ for the full parameter space or random A with $r = 100$, at scales 10^{-3} and 5×10^{-4}. Large-batch solutions generally have higher scores and lower test accuracy. Starting large-batch training from small-batch checkpoints can reduce sharpness and improve test accuracy, suggesting that the training trajectory also matters.

2 Reproduction of Figure 3

I used ChatGPT, model Astra, to discuss the paper and review a proposed PyTorch notebook. The shared conversation records the model identifier `gpt-6-astra-wm`.
Conversation: `https://chatgpt.com/share/6adab3d8-c2fc-83ea-8a39-08fb4b3fff4c`
Notebook: `on_large_batch_training_for_deep_learning.ipynb`
GitHub repository: `https://github.com/Jireh1226/AdvancedML_assignements`
Submission folder: `Assignment_2/paper_2_large_batch/`.

The executed notebook and all six runs were produced on 22 October with Codex/IDE assistance, before the shared chat that evening. The chat contains a later paper discussion and a new notebook proposal that was never executed. The reproduced figure comes from the earlier notebook, so this workflow does not meet the assignment's chat-interface requirement. The phase headings below organise the discussion and evidence; they do not describe the order in which the work occurred.

2.1 Phase 1: understanding the paper and asking for code

I summarised my understanding and asked the LLM to check it against the paper. The discussion clarified that sharpness concerns sensitivity to changes in the parameters, that Figure 3 evaluates already-trained models, and that a sharp solution need not have steep loss in every direction. I then requested F1, C1, and C2 using only the supplied paper, with missing settings and proposed assumptions identified before implementation. Existing implementations were excluded from that request.

2.2 Phase 2: reviewing the LLM's output

The generated notebook included lengthy instructions and experiments for Table 2 and Tables 3–4, beyond the three assigned Figure 3 panels. A request to shorten it removed instructions but left the extra sharpness experiments. The LLM later agreed to remove them. They were never run.

The main numerical concern was how to handle BN statistics along the weight path. The LLM initially considered affine interpolation of stored variances, then recognised that extrapolation could make them negative. It proposed geometric interpolation instead:

$$v_\alpha = \exp((1-\alpha)\log v_{\mathrm{SB}} + \alpha \log v_{\mathrm{LB}})\,.$$

This keeps variances positive and preserves the endpoint models, but gives no evidence that the variances match activations at intermediate weights. It remains untested; the saved checkpoints used training-data recalibration.

C1 used asymmetric SAME padding in the proposal. For a 32-pixel input, five-pixel kernel and stride two, padding one pixel before and two after gives 16 outputs. Symmetric padding of two gives the same count but shifts the receptive fields by one pixel. The paper leaves this allocation unspecified; the executed model retained symmetric padding.

The proposed training loop also differed from the executed one. It checked loss every epoch instead of every five epochs, used patience 20 instead of 30, and omitted the training-accuracy and loss requirements. It also dropped incomplete batches, which the LLM justified using the paper's fixed-batch-size description. The paper does not specify whether to discard these batches.

The proposed five seeds follow the paper; the six completed models cover one seed, leaving run-to-run variation unmeasured. The proposal also uses FP64 and 121 path points without justification, compared with FP32 and 61 points in the executed notebook.

The proposal stores a fixed implementation hash without checking the running code. The LLM acknowledged that it could remain unchanged after edits and provides no runtime integrity check.

2.3 Phase 3: running, correcting, and comparing the results

2.3.1 Settings used in the attempt

F1 used all 60,000 MNIST training images and 10,000 test images. Its five hidden layers each contain 512 units. C1 and C2 used all 50,000 CIFAR-10 training images and 10,000 test images. C1 contains two 64-filter, 5×5, stride-two convolution stages, each followed by 3×3 max pooling, then hidden dense layers of 384 and 192 units. C2 uses convolution groups of depths $(2, 2, 3, 3, 3)$ and widths $(64, 128, 256, 512, 512)$, pooling after each group, and a 512-unit hidden dense layer. All networks have ten output logits.

Hidden affine layers are followed by BN and ReLU. C1's two dense hidden layers and C2's dense hidden layer use dropout probability 0.5; F1 has no dropout. BN is omitted from the output logits. The parameter counts are 1,462,794 for F1, 283,274 for C1, and 14,991,946 for C2. C1's symmetric convolution padding is two; pooling has stride two and padding one. The final C1 feature map is $64 \times 2 \times 2$.

All pairs used seed 42 and identical initial weights, verified by saved initial-state hashes. Xavier-uniform weights and zero biases were used, with BN scale one, shift zero, numerical stabiliser 10^{-5} and momentum 0.1. The paper does not report seeds; 42 is a repeatability choice.

Stated in the paper	Choice in the executed reproduction
Adam; SB 256; LB 10% of training data	Learning rate 0.001; coefficients $(0.9, 0.999)$; stabiliser 10^{-8}; no schedule or weight decay
Appendix B network sizes, BN and dense-layer dropout	Hidden-layer BN before ReLU; symmetric C1 padding; Xavier-uniform initialisation; paired seed 42
Training until loss ceases improving; five trials	One trial; checks every five epochs; minimum 100 and cap 300; explicit plateau and fitting conditions
Linear weight path over $[-1, 2]$	61 points, spacing 0.05; train-only BN recalibration at every point; FP32 network evaluation

Table 1: Paper-specified settings and implementation assumptions. Later code inspection is reported separately in Phase 4.

Images were scaled to $[0, 1]$, retaining their original dimensions, without augmentation or further standardisation. Cross-entropy was computed from raw logits and averaged over examples. Each large batch was one forward/backward batch: 6,000 images for F1 and 5,000 for C1/C2. Accumulating gradients over smaller batches would change BN behaviour. The final incomplete small batch was included, giving 235 updates per MNIST epoch and 196 per CIFAR-10 epoch, compared with ten for either large-batch dataset.

The minibatch averaging and Adam updates follow Lecture 6, pages 38–42, and Lecture 6.1, pages 23–24, respectively. The numerical settings chosen for this implementation are listed separately from the paper's specifications in the table.

Training stopped after at least 100 epochs when the full-training evaluation loss had not beaten its recorded reference by 10^{-4} for at least 30 epochs, provided training accuracy was at least 99% and loss was below 0.1. Test accuracy did not control stopping. All six runs met this rule. C1 large-batch met it at the 300-epoch cap; the paper describes training without a fixed budget.

At each path point, only trainable parameters were interpolated. Every BN layer's running statistics were reset. Dropout stayed off, while BN alone used training mode for a pass through training images in batches of 512. For a batch of m images after c images had been processed, the update momentum was $m/(c + m)$. The resulting statistics were then frozen for evaluation. These are approximate statistics: deeper activations depend on the calibration batches, and averaging

batch variances differs from computing variance over all activations. No test images or labels entered calibration. Recalibration can change endpoint predictions even though the endpoint trainable weights match exactly.

2.3.2 Checks and changes before execution

A memory-saving C2 implementation was tested with safeguards against updating BN statistics twice during activation recomputation. Its gradients and BN buffers matched ordinary execution in the CPU check. The genuine 5,000-image batch also fitted ordinary FP32 GPU execution, which was faster in the pilot, so the recomputation code was removed before the full runs. Dataset loading was made self-contained through the standard MNIST and CIFAR-10 downloaders. These changes did not alter the training data or batch sizes.

Model dimensions, paired initialisation, finite losses and checkpoint continuation were also checked before training.

2.3.3 Results and differences from the paper

Network	Epochs SB/LB	Reproduced SB	Reproduced LB	Paper SB	Paper LB
		Test accuracy (%)			
F1	100/100	98.61	98.20	98.03 ± 0.07	97.81 ± 0.07
C1	165/300	71.00	66.78	80.04 ± 0.12	77.26 ± 0.42
C2	165/180	86.07	81.33	89.24 ± 0.12	87.26 ± 0.07

Table 2: Reproduced path endpoints after train-only BN recalibration, compared with Table 2's five-trial means and standard deviations. The reproduction has one trial. Table 2's means are not the exact pair displayed in the original Figure 3.

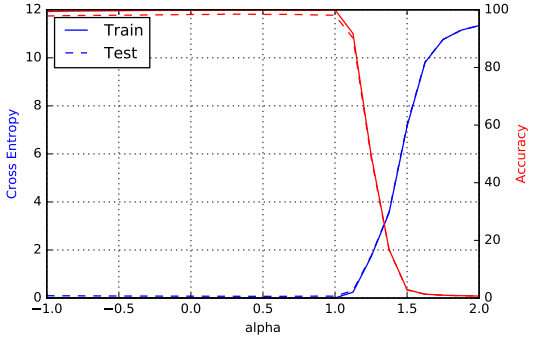
The recalibrated training accuracies are 99.96%/100% for F1, 99.82%/99.89% for C1, and 99.84%/100% for C2, ordered SB/LB. All three small-batch endpoints have higher test accuracy than their large-batch counterparts. Their test gaps are 0.41, 4.22 and 4.74 percentage points. C1 is 9.04/10.48 points below the paper's means, and C2 is 3.17/5.93 points below them. F1 is slightly above the reported means.

Figure 1 compares the original and reproduced panels. Both show a low-training-loss region around the small-batch endpoint and a steeper rise beyond the large-batch endpoint. C1 and C2 also show a barrier between the solutions. The agreement concerns the direction evaluated under the chosen BN convention.

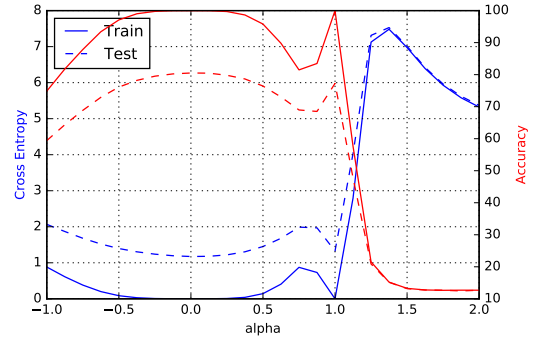
F1 has the clearest shape mismatch. In the paper, its accuracy falls towards zero beyond the large-batch endpoint and stays low, while cross-entropy rises towards approximately 11. In the reproduction, the accuracy drop occurs closer to $\alpha = 1$, test accuracy reaches 2.20% at $\alpha = 1.3$, then recovers to 27.91% at $\alpha = 2$. Test cross-entropy falls from 7.10 to 4.55 over those two points.

C1 retains the barrier and sharp post-LB rise, but its test-accuracy curve is lower and its extrapolated cross-entropy peaks near 14 rather than about 7.5 in the paper. C2's endpoint pattern is similar, but its reproduced test loss reaches 12.62 at $\alpha = 2$, compared with roughly 6 in the original plot. Its accuracy also drops earlier and settles around 13.65%, above the paper's right-hand tail. The original values in this paragraph are visual approximations; the reproduced values come from saved measurements.

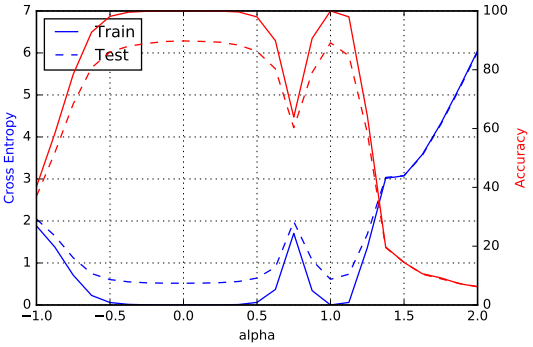
Original Figure 3



(a) F_1

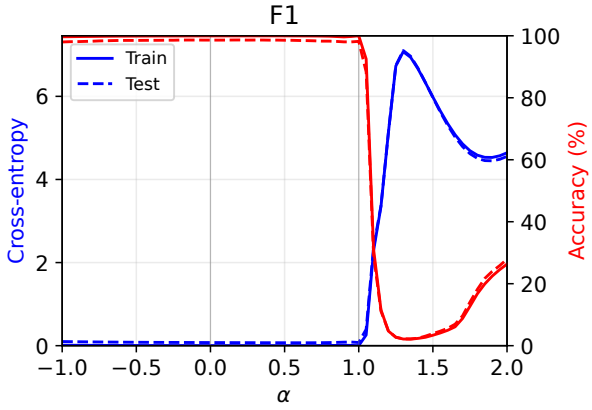


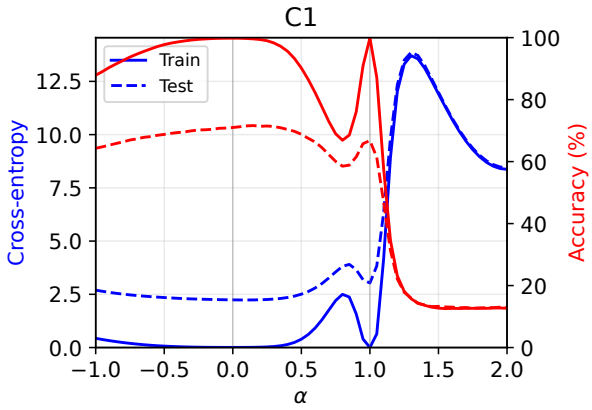
(c) C_1



(d) C_2

Reproduction, seed 42





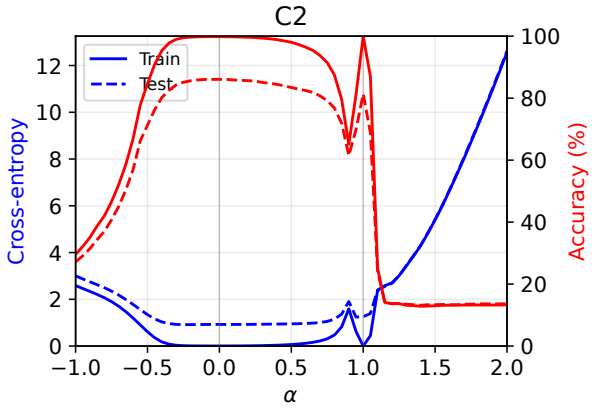


Figure 1: F1, C1 and C2, from top to bottom. Original panels are cropped from the supplied paper [1]; reproduced panels display all 61 saved points with larger labels and no smoothing or altered measurements. Blue is cross-entropy and red is accuracy; solid is training and dashed is testing. Each panel uses its own loss scale.

2.3.4 Diagnosing the F1 rebound

The F1 rebound prompted a check of BN handling. Recalibration was a suspected cause, so the saved F1 checkpoints were evaluated with alternative BN treatments while keeping the trainable weights at each α unchanged. First, repeating the original calibration procedure reproduced all 61 saved training and test points exactly.

BN treatment	$\alpha = 1.3$	$\alpha = 2$
	F1 test accuracy (%)	
Training-data recalibration	2.20	27.91
Fixed small-batch statistics	10.10	11.12
Fixed large-batch statistics	3.26	18.04
Affine stored-variance extrapolation	Invalid	Invalid

Table 3: Completed evaluation-only controls. Geometric variance interpolation from the later chat proposal is not included because it has not been tested.

Affine extrapolation produced negative running variances at every sampled point from $\alpha = 1.05$ to 2. At $\alpha = 1.3$, all 2,560 variance entries were negative, so this treatment was rejected without clamping them or assigning accuracies. Fixed large-batch statistics still produced a rebound, showing that it can occur without recalibration. Fixed small-batch statistics largely collapsed predictions to digit 3. Both fixed-statistics controls preserve the complete model at only one endpoint.

Under recalibration, the rebound reflects partial recovery on digits 0, 3, 4 and 9. A rise in accuracy can also accompany a rise in cross-entropy: with fixed LB statistics, test accuracy rises from 3.26% to 18.04% while cross-entropy increases from 66.83 to 56,913.09. Very confident errors can dominate the loss even when more predictions are correct. BN handling affects the curve, but these controls leave the cause of the difference from the authors' selected pair unresolved.

2.3.5 Other checks and remaining limitations

Reloading all six checkpoints reproduced the saved losses and accuracies at $\alpha = 0, 1, 1.2$ exactly. Recalibrating with batches of 256 instead of 512 changed test accuracy by at most 0.02 percentage points across those nine points, and cross-entropy by at most 0.00959. The effect of changing the calibration batch size is too small to explain the endpoint accuracy gaps.

The raw running-BN test accuracies before path recalibration were 98.68%/98.22% for F1, 69.56%/65.67% for C1, and 85.36%/81.35% for C2. These are distinct from Table 2. C1 SB peaked at 71.67% at epoch 30 and finished at 69.56%; C1 LB peaked at 68.33% at epoch 30 and finished at 65.67%. Both therefore show some decline during continued training.

The reproduced paths show better SB test performance and a steeper loss rise beyond LB, with different loss heights and curve shapes. Phase 4 identifies differences in C1/C2 BN evaluation, C1 stride and C2 dropout. Their effects, those of stopping and initialisation, and variation across seeds remain unmeasured. The released code contains no F1 implementation from which to recover that panel's evaluation convention.

2.4 Phase 4: comparison with the released implementation

The executed notebook matches the local Phase 3 snapshot at commit `8b28d5b`. The change log records author-code inspection after that snapshot, but the exact initial inspection time was not preserved. The file `phase4_record.json` records the notebook hash, source versions, files read and a timestamped recheck. The comparison used the authors' repository at commit `8819876` [3].

Its Keras/Theano scripts cover C1–C4, with no F1 implementation. The separate PyTorch folder provides a preliminary VGG11-based example rather than the assigned three models.

The Keras script agrees on scaling pixels by 1/255, using Adam, and batches of 256 and 5,000. It saves and restores the same initial weights between SB and LB and resets the optimiser. This controls the starting weights while changing batch size, agreeing with the paired initialisation used here. However, it trains for 20 epochs and evaluates 25 path points, whereas the paper describes loss-based stopping without a budget. The released script and paper therefore specify different training procedures.

Two architecture differences are substantial. The released C1 convolutions omit a stride argument; the historical Keras API defaults to stride one [4]. Appendix B.3 specifies stride two, which the reproduction follows. The released version consequently has a larger feature map before its dense layers. The released C2 also adds dropout with probabilities 0.3 and 0.4 within convolution groups and dropout before the dense layer, beyond the dense-layer dropout described in the paper and implemented in the reproduction.

BN evaluation also differs. The released networks use Keras BN `mode=2`, which computes statistics from each input batch during both training and testing [4]. The plotting script evaluates in batches of 5,000, including test batches, and interpolates only trainable weights. The reproduction instead estimates running statistics on training images and freezes them for testing. The effect of using the released convention remains unmeasured; the fixed-buffer controls and geometric-variance proposal use different procedures.

A next check would evaluate the saved C1/C2 paths with input-batch BN statistics, batches of 5,000 and dropout disabled. Keeping weights fixed and matching the historical normalisation details would isolate the evaluation change. C1's stride and C2's extra dropout require separate paired training runs. Each change should be tested against the paper-based configuration.

These are proposed controls, not completed corrections. The comparison was limited to reading the code. No author code was executed or copied into the notebook, and the checkpoints and primary figures remained unchanged.

2.5 Code and interaction record

The shared chat reports synthetic software checks but no full MNIST/CIFAR-10 training.

The local results are in `results_large_batch/seed42_v1/`; separate BN controls are in `results_large_batch/audit_f1_bn/`. Settings, histories, checkpoint identities and failed proposals are recorded in the assignment-root `reproduction_changes.md`. The report's enlarged panels are generated from saved measurements by `report_assets/render_panels.py`, with source hashes recorded alongside them.

3 Critical review

3.1 Assessment of the paper

The paper's main strength is measuring loss geometry separately from test accuracy across several networks and datasets. This lets the authors compare the shape of the loss near the solutions with their performance on unseen data.

The starting point also matters. Each setup was repeated from five random initialisations, and the authors mention similar behaviour under other initialisation strategies, although they give little detail about those alternatives [1]. Table 2 shows lower mean large-batch test accuracy on all six networks; it does not establish that every individual run was worse. In Figure 5, starting large-batch

training from later small-batch checkpoints improves test accuracy and reduces sharpness compared with earlier checkpoints. These results show that the gap depends on the training history. A systematic comparison of named initialisation schemes across paired seeds would help distinguish sensitivity to random starting weights from the benefit of preliminary small-batch training.

Several effects change alongside gradient noise when batch size changes. These include the number of updates per epoch and the statistics used by BN. Separating these effects would help test whether sharpness causes the performance gap. Further experiments could compare equal-example and equal-update budgets, control the normalisation convention, and test whether adding noise to large-batch updates changes sharpness and test accuracy together across several seeds.

The noise explanation also needs to account for Adam's memory of previous gradients. Lecture 6.1, pages 5–7 and 23–24, describes momentum and Adam's first- and second-moment estimates [2]. Momentum can sustain movement when the current gradient is small, but provides no general escape guarantee: with a zero gradient estimate and zero first-moment state, Adam takes no step. Both batch regimes in the paper already use Adam, and the authors report similar large-batch findings with other optimisers. A controlled comparison across momentum coefficients, with matched update budgets and learning-rate tuning, would help test how momentum affects the gap.

Lecture 4, pages 19–27, gives a way to connect sharpness to the rate at which the loss gradient changes [2]. Assuming a Lipschitz-continuous gradient within a neighbourhood gives

$$\|\nabla L_S(u) - \nabla L_S(v)\|_2 \leq L_\nabla \|u - v\|_2.$$

Where the loss is twice differentiable, L_∇ bounds the Hessian's operator norm. A direction of high positive curvature requires a larger bound, linking this condition to the Hessian discussion in Lecture 6.1, pages 9–11. For ReLU networks, this reasoning applies within smooth regions; global smoothness is not assured. The condition concerns sensitivity of the training loss, so a claim about generalisation still requires evidence on unseen data.

Figure 3 measures curvature along only one direction. For a fixed evaluation convention and $d = w_{\mathrm{LB}} - w_{\mathrm{SB}}$, the path's local second derivative is $d^\top \nabla^2 L_S(w) d$ wherever the loss is twice differentiable. Its apparent steepness depends on both the direction and its length. A flat section can coexist with sharp directions elsewhere, so this slice cannot determine a Lipschitz-gradient bound over the whole neighbourhood. The full-space and random-subspace searches partly address this limitation, although they remain approximate and depend on the permitted perturbation region.

Sharpness also depends on how the parameters are represented. In a ReLU network, multiplying a hidden unit's incoming weights and bias by a positive constant and dividing its outgoing weights by that constant can preserve the predictions while changing parameter-space distances and curvature. The paper's coordinate-scaled perturbation box addresses differences in weight magnitude, but does not establish invariance under all function-preserving changes. A comparison using controlled rescaling would help separate sensitivity of the predictor from sensitivity of its parameterisation.

The released C1/C2 code makes each prediction depend on the other images in its evaluation batch through BN statistics. The measured path therefore depends on evaluation batch size and composition as well as the weights. Comparing BN conventions would help separate evaluation effects from training effects. The stride and dropout discrepancies also leave uncertainty about which settings generated the published panels.

Exact padding, numerical optimiser settings and an operational stopping rule also need clarification. The five-trial summaries provide evidence beyond a single run, but Figure 3 displays one selected pair and does not show how much its shape varies across trials. Reporting all five paths with the exact run configuration would make the empirical claim easier to assess and reproduce.

Appendix B.4 also names C3 when introducing the deep ten-output architecture. The heading, Table 1 and surrounding text identify it as C2; C3 is the shallow CIFAR-100 model described in

B.3. This appears to be a naming error rather than a different architecture, but it adds avoidable ambiguity for reproduction.

The authors also argue that their generalisation gap differs from the decline in test performance associated with continued training. Their displayed learning curves support that distinction. The reproduced C1 small-batch run did show such a decline, so excluding this explanation depends on the training setup.

3.2 Comparison with the OpenReview discussion

The public reviews also value the empirical contribution. Reviewer 1 emphasised its usefulness, while Reviewer 2 considered the evidence valuable despite limited novelty [5].

Reviewer 3 asked whether the endpoints were genuine local minima or only minima along one-dimensional slices. This overlaps with the concern above about interpreting a single path. The authors acknowledged that second-order optimality had not been checked, leaving uncertainty about the endpoints themselves as well as the directions examined around them.

Reviewer 2 challenged the claimed invariance to loss rescaling because the sharpness denominator contains $1 + L$. The authors withdrew that claim and reported similar results with an alternative denominator. That objection concerns changing the loss scale. The parameter-rescaling concern raised here concerns changes to the weights that preserve the network's predictions.

Reviewer 3 and public commenter Alex Lamb suggested noise injection. The authors reported no consistent test-error improvement from their attempted noise variants, leaving the role of noise unresolved. The update-budget and BN controls proposed here would help test the other effects of changing batch size.

References

[1] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *International Conference on Learning Representations*, 2017.

[2] Hirak Sarkar. *CS 6362 Advanced Machine Learning lecture slides*, Fall 2026. Supplied files `Lecture_4.pdf`, pp. 19–27; `Lecture_6.pdf`, pp. 38–42 and 50; and `Lecture_61.pdf`, pp. 5–7, 9–11 and 23–24.

[3] Nitish Shirish Keskar et al. *Large-batch training: released experiment code*. Repository at commit 8819876. Accessed 22 October 2026; file identities and recheck times are in `phase4_record.json`.

[4] Keras team. *Keras 1.2.2 historical layer implementations*. Batch normalisation and convolution layers. Used to interpret the released Keras 1.x API, not to establish the authors' exact installed version.

[5] OpenReview. Official reviews, author responses, and public discussion for paper `H1oyRlYgg`. Original discussion; accessed through archived OpenReview records, 22 October 2026. Reviews `S1Eq3CUEl`, `SJVFyZfNe`, `HkYQOlGEl`; questions and replies `ByV6OKkQg`, `HkcS-1U7l`, `HJWN7OkXe`, `SJdX9TrQg`, `H1UEbWm4x`, `SJMMtBgrg`, `H1oYDrgSe`.